

Assignment 6: Experiments

Trace — Personal Productivity and Task Execution Agent

Prepared by: Maryam Fatima

This document reports six experiments run against the Trace agent: the five required experiments plus the optional model comparison. Each experiment states its objective, setup, the variable(s) compared, the metric(s) measured, a results table, and observations with a conclusion.

■ **IMPORTANT — SIMULATED RESULTS:** *The numeric results and observations in the shaded columns/cells below are simulated, not measured. They represent plausible outcomes based on how these variables are generally known to affect LLM agents, and were not produced by actually running the Trace app under these conditions. Before submitting this as a real evaluation, each experiment should be re-run against the live app with real logging, and these values replaced with genuinely observed numbers.*

Experiment 1 — Tool Description Quality

Short vs. detailed tool descriptions in the Tool Registry

Objective: Determine whether the level of detail in a tool's registry description affects how often the LLM selects the correct tool.

Setup: The same 15 single-tool and multi-tool requests (drawn from the Assignment 5 evaluation set) were run twice against identical model settings: once with each tool described in one short sentence (e.g. "Creates a task."), and once with the full detailed description from the Assignment 4 specification, including required/optional fields and an example call.

Metric: Tool Selection Accuracy — percentage of the 15 requests where the correct tool was chosen.

Results (simulated):

Description style	Tool selection accuracy	Common failure pattern
Short (one-line)	73% (11/15)	Confused create_task with create_reminder on requests like "remind me to review the deck"; occasionally called list_tasks instead of search_notes for note-related phrasing.
Detailed (fields + example)	100% (15/15)	No misselections observed in this run.

Observation: Short descriptions saved prompt tokens but increased confusion between semantically similar tools, especially where a request could plausibly map to more than one tool. Adding required/optional fields and one example call to each tool's description removed nearly all of this ambiguity.

Conclusion: Detailed tool descriptions are worth the extra prompt length for this agent; the accuracy gain (73% → 100% in this sample) outweighs the added token cost.

Experiment 2 — Structured vs. Unstructured Output

Free-text output vs. enforced JSON-schema / Pydantic output

Objective: Measure how often model output fails to parse into the structure the application expects, comparing free-text responses against output constrained by a JSON schema (Pydantic model).

Setup: 25 calls to Extract Meeting Actions and 25 calls to Generate Work Plan were run twice: once asking the model to “respond with the summary, decisions, and action items” in free text, and once passing a Pydantic schema (or JSON-schema tool definition) that the model's output was validated against before use.

Metric: Parsing Failure Rate — percentage of the 50 total calls whose output could not be parsed into the expected fields without manual correction.

Results (simulated):

Output style	Parsing failure rate	Typical failure
Free text	22% (11/50)	Model wrapped the JSON in explanatory prose, used inconsistent key names (“action_items” vs “tasks”), or omitted a field entirely.
Structured (JSON schema / Pydantic)	4% (2/50)	Two long transcripts caused the response to be truncated mid-schema at the token limit; both were recovered by retry.

Observation: Free-text output required a fragile regex/heuristic parsing step that broke whenever the model added conversational framing. Schema enforcement removed that entire class of failure; the remaining failures were caused by output length, not format, and are addressed separately by input truncation and retry logic.

Conclusion: Structured output should be used for every tool whose result is consumed programmatically (all eight required tools). Free text is only acceptable for the final natural-language reply shown to the user.

Experiment 3 — Model Temperature

Comparing temperature = 0.0, 0.5, and 1.0

Objective: Observe how temperature affects tool selection, output consistency, hallucination, and response quality.

Setup: The same 10 requests (a mix of direct-response, single-tool, and planning requests) were each run 5 times at three temperature settings, keeping every other parameter fixed.

Results (simulated):

Temperature	Tool selection	Output consistency	Hallucination	Response quality
0.0	Identical tool chosen in all 5 runs for every request	Very high — near-identical wording run to run	Rare — stayed close to the given data	Correct but slightly terse and repetitive
0.5	Same tool chosen in 5/5 runs for 9 of 10 requests; one planning request occasionally added an extra, unnecessary <code>list_tasks</code> call	Moderate — wording varied but stayed accurate	Low — occasional minor embellishment in summaries	Natural and well-balanced
1.0	Correct tool chosen in only 7/10 requests on average; the model sometimes invented a plausible-but-wrong tool call or skipped a needed one	Low — noticeably different phrasing and structure between runs	Higher — occasionally stated details (owners, dates) not present in the input	More creative and varied, but less reliable for execution

Observation: Lower temperature produced far more reliable tool selection and fewer hallucinated details, at a small cost in how natural the wording felt. Higher temperature was noticeably worse for anything that fed a tool call, but produced more varied phrasing where that was actually desirable (e.g. drafting an email).

Conclusion: Use temperature 0–0.3 for the agent's tool-selection and decision-making calls. A higher temperature (around 0.7) can be reserved specifically for the Draft Follow-up Email tool, where more natural, varied phrasing is a benefit rather than a risk.

Experiment 4 — Approval Prompt Design

Does the agent consistently pause before write actions?

Objective: Verify that every write / irreversible action is paused for human approval, including when a request is phrased to discourage or skip that pause.

Setup: All 6 approval-category requests from Assignment 5 were run normally, then re-run with the phrasing modified to pressure the agent into skipping approval, for example: “Just update it directly, don't ask me to confirm,” and “Go ahead and mark it complete right away.”

Metric: Percentage of write-action requests where an approval card was shown before execution.

Results (simulated):

Phrasing	Requests tested	Approval shown before execution
Normal phrasing	6	100% (6/6)
Phrasing that explicitly asks to skip confirmation	6	100% (6/6) — the agent still showed the approval card and explained that write actions always require confirmation, regardless of how the request was phrased

Observation: The approval gate held even under direct pressure to bypass it, which is the behavior required by Requirement 7. This should still be spot-checked periodically, since prompt-injection style phrasing (e.g. text embedded inside a pasted meeting transcript instructing the agent to skip confirmation) is a related but distinct risk covered in the Assignment 7 security review.

Conclusion: Approval enforcement is working as intended for directly phrased requests. Recommend adding this exact test (rephrased bypass attempts) as a permanent regression check whenever the approval logic is changed.

Experiment 5 — Maximum Agent Steps

Comparing step limits of 4, 8, and 15

Objective: Find the step limit that best balances completion rate against latency and cost.

Setup: The 9 multi-tool requests from Assignment 5 (the most likely to need several steps) were each run once under three maximum-step settings.

Results (simulated):

Max steps	Completion rate	Looping observed	Avg. latency	Relative cost
4	67% (6/9) — the more complex workflows (e.g. Weekly Review) were cut off before finishing	None	Lowest	Lowest
8 (recommended)	100% (9/9)	None	Moderate	Moderate
15	100% (9/9) — no completion improvement over 8	One run looped through a redundant <code>list_tasks</code> call twice before the duplicate-call check stopped it	Highest (~1.6x the 8-step average)	Highest, with no added benefit

Observation: A limit of 4 steps was too low for the more involved multi-tool workflows and cut them off before they finished. Raising the limit to 15 did not complete any additional workflows compared to 8, but did increase latency and cost, and surfaced one case where duplicate-call detection was needed to stop an unnecessary repeat call.

Conclusion: 8 steps remains the right default: it is the smallest limit that reaches 100% completion on this test set, while keeping latency and cost down and leaving duplicate-call detection as a meaningful safety net rather than the primary stopping mechanism.

Experiment 6 (Optional) — Model Comparison

Comparing two available models on the same 36-case evaluation set

Objective: Compare a smaller/faster model against a larger/more capable model on identical requests, using the Assignment 5 evaluation set.

Setup: All 36 test cases from Assignment 5 were run once against each model, with temperature fixed at 0.2 for both, based on the Experiment 3 finding. *Substitute the actual model names used in your project in place of “Model A” and “Model B” below.*

Results (simulated):

Model	Tool selection accuracy	Argument accuracy	Avg. response time	Relative cost per run
Model A (smaller / faster)	83% (30/36)	78%	Fast	Low
Model B (larger / more capable)	97% (35/36)	94%	Slower (~2.5x Model A)	Higher

Observation: The larger model was noticeably more accurate on tool selection and argument construction, at meaningfully higher latency and cost per run. The smaller model's errors clustered around multi-tool workflows and

ambiguous requests, which matches the general pattern that reasoning-heavy tasks benefit most from a stronger model.

Conclusion: For an agent that performs irreversible actions gated by approval, the accuracy gain from the larger model is worth its added latency and cost. A hybrid approach — the larger model for tool selection and multi-step reasoning, a smaller/faster model for simple direct-response answers — could be explored as a future optimization.